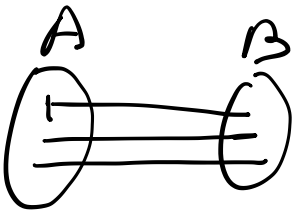
2016-17 Final

1:1 maping :- Where a entity A with at most one entity in B, an entity B is associated with A



Example: Passport and driving license

pose there are two relations:

- employees(ID, Name, DOB, DeptName, Dept Budget, Dept Location).
- Students (ID, Name, DOB, DeptName, Dept Building Area Size (in squire meter))

e tasks are as follows:

i) Criticize the design with its major pros and cons.
ii) Propose an alternative design to eliminate the problems you have just mentioned.
iii) Finally place proper examples to prove that your modified design is able to answer following queries:

- Find the students' name and date of birth who study in the departments that are located at the "Second Academic Building".
- Find the employees name and date of birth who work in the departments that are bigger than 4500 squire meter in size.

i) Critique of the Given Design:

Pros:

1. **Simplicity:** The design is straightforward, with only two tables.
2. **Common Fields:** Both tables have common fields like ID, Name, and DOB, which can simplify queries involving both employees and students.

Cons:

1. **Redundancy:** The design is redundant as it duplicates information about the department (Dept, Dept Budget, Dept Location, Dept Building, Area Size) in both tables. If department details change, it needs to be updated in two places, increasing the risk of inconsistencies.
2. **Lack of Normalization:** The design violates normalization rules, leading to data redundancy and potential update anomalies.
3. **Limited Extensibility:** If there are more entities related to the department in the future (e.g., courses, projects), extending the schema would be complex and inefficient.
4. **Maintainability:** Due to redundancy, maintaining data integrity and consistency across the tables can be challenging.

ii)

Proposed Alternative Design:

To eliminate the problems mentioned above, a better approach would be to normalize the schema and eliminate redundancy. Here's the proposed design:

1. **Employee Table:**
   - EmployeeID (Primary Key)
   - Name
   - DOB
   - DeptID (Foreign Key referencing Department table)
2. **Student Table:**
   - StudentID (Primary Key)
   - Name
   - DOB
   - DeptID (Foreign Key referencing Department table)
3. **Department Table:**
   - DeptID (Primary Key)
   - DeptName
   - DeptBudget
   - DeptLocation
   - DeptBuilding
   - AreaSize

iii) Example :-

Employee Table: (e)

EmployeeID	Name	DOB	DeptID
1	Alice	1990-05-15	1
2	Bob	1985-08-23	2
3	Charlie	1988-12-10	1

Student Table: (s)

StudentID	Name	DOB	DeptID
1	David	1992-03-20	2
2	Emma	1991-07-08	1
3	Frank	1993-09-25	3

Department Table: (d)

DeptID	DeptName	DeptBudget	DeptLocation	DeptBuilding	AreaSize
1	IT Dept	500000	2nd academic bldg	A	4000
2	Math Dept	600000	Main Building	B	5000
3	Eng Dept	700000	2nd academic bldg	C	6000

Query result

a) Select S.name, S.DOB
From Students Join Dept d,
ON S.DeptID = d.DeptID
WHERE d.DeptLocation
= '2ndAB';

Result :

b) SELECT e.name, e.DOB
From employee e JOIN
Dept d
ON e.deptID = d.DeptID
WHERE d.Areasize > 4500;

Result :

Some terms with Examples Q(2 → i)

1. Projection:

Definition: Projection is the operation in relational algebra that retrieves specific columns from a table while discarding the rest of the columns.

Example:
Consider a table `Students` with columns `StudentID`, `Name`, `Age`, and `Grade`. A projection query to retrieve only `StudentID` and `Name` would be:

```sql
SELECT StudentID, Name FROM Students;
```

2. Selection:

Definition: Selection is the operation in relational algebra that retrieves specific rows from a table based on a given condition.

Example:
Using the same `Students` table, a selection query to retrieve students with `Grade` 'A' would be:

```sql
SELECT * FROM Students WHERE Grade = 'A';
```

3. Cartesian Product:

Definition: The Cartesian Product of two tables results in a combination of all rows from the first table with all rows from the second table.

Example:
Consider two tables `TableA` with columns `A` and `B` and `TableB` with columns `X` and `Y`. The Cartesian Product of these tables would be:

```sql
SELECT * FROM TableA, TableB;
```

4. Super Key:

Definition: A super key is a set of one or more attributes (columns) that can uniquely identify a tuple (row) in a table.

Example:
In a table `Employees` with columns `EmployeeID`, `SSN`, and `Email`, the combination of `EmployeeID` and `SSN` can be a super key because it can uniquely identify each employee.

5. Candidate Key:

Definition: A candidate key is a minimal super key, meaning it is a super key without any unnecessary attributes.

Example:

In the `Employees` table, if both `EmployeeID` and `SSN` are unique, they are candidate keys.

6. Primary Key:

Definition: A primary key is a candidate key selected as the main method of uniquely identifying records in a table. It must be unique for each record and cannot contain NULL values.

Example:

In the `Employees` table, if `EmployeeID` is chosen as the primary key, it uniquely identifies each employee and cannot contain duplicate or NULL values.

7. Foreign Key:

Definition: A foreign key is a field in a table that is used to establish a link between the data in two tables. It points to the primary key of another table and helps establish relationships between tables.

Example:

Consider a `Orders` table with a foreign key `CustomerID` referring to the `CustomerID` primary key in a `Customers` table. This establishes a relationship where each order is associated with a specific customer.

Regenerate

ER Diagram এর Q গুলো Solve করো.

করো → ?? → পারলে না করো!!

Q(2 → b)

↳ পরে কেন করলে

2(c) Inner vs Outer Join

Inner Join:

- **Returns:** Only the matching rows from both tables.
- **Excludes:** Rows with no matching counterpart in the other table.
- **Result:** Contains only common rows between the tables.

Outer Join (LEFT, RIGHT, FULL):

- **Returns:** All rows from at least one of the tables.
- **Includes:** Rows with no matching counterpart in the other table.
- **Result:** Contains both common rows and unmatched rows from one or both tables.

3(a) DDL, DML

DDL → is used for defining and managing the structure of database

↳ create, ALTER, DROP

DML → used for managing data within database

↳ select, insert, update, delete

(3b) → i) Create Table

Table: Department		
Attribute	Description and/or Data Type	Requirement / Other information
ID	Numeric with no decimal part.	Primary key
Short Name	3 characters	Exactly 3 characters, can not be null
Full Name	100 characters	Can be null
Date of Establishment	Date	Can be null
Location	35 characters	Value should be either i) First Academic Building or ii) Second Academic Building
Head of Dept	Present head of department	Foreign key referencing the table Teachers

```
create TABLE Dept(
    ID INT Primary key,
    ShortName varchar(3) not null,
    FullName varchar(100),
    DateofE Date,
    Location varchar(35) check(Location IN
        ('1stAB','2ndAB')),
    HOD varchar(20),
    Foreign key(HOD) References Teachers(t_id)
```

Table: Teachers

Attribute	Description and/or Data Type	Requirement / Other
ID	Numeric with no decimal part.	Primary key
Name	50 characters	It may be used by other than English language such as Chinese.
DOB	Date	No date before January 1, 1950 is allowed.
Designation	25 characters	Any values from the List: {Lecture, Assistant Professor, Associate Professor, Professor}
Salary	Monthly Basic Salary	May be null only for unemployed.
BloodGroup	Character	Any value from List{A+ve, A-ve, B+ve, B-ve, AB+ve, AB-ve}
mediate Boss		Foreign Key referencing Teachers table itself (self-reference).

```
Create Table Teachers(
T_ID INT Primary key,
Name Varchar(50),
DOB Date,
Designation varchar(25) check(Designation IN
('Lect', 'Assi. Prof.', 'Asso. Prof', Prof')
Salary number(10,2),
Blood varchar(5) check(Blood IN
('A+', 'B+'_ _ _ _ _ )
```

Imm.Boss int ,

Foreign key (immBoss) References teachers(t_ID)

Table: Students

Attribute	Description and/or Data Type	Requirement / Other information
ID	Numeric	Primary key, E.g. 164409
Name	60 characters	
DOB	Date	It must be greater than 31-DEC-1985
Dept	Student's dept.	Foreign key referencing the table **Department**

Table: Courses

Attribute	Description and/or Data Type	Requirement / Other information
ID	8 characters	Primary key, E.g. CSE 4401
Title in Full	60 characters	
Credit Hour	Numeric	It must be between 0.5 to 4.0
Offering Dept	Which dept is offering the course	Foreign key referencing the table **Department**

```
create Table Student(
    ID numeric(10,0),
    Name varchar(20),
    DOB DATE check(DOB > 31-DEC-1985),
    Dept varchar(20),
    Foreign key (Dept) References Dept(Dept_id )
```

```
create Table Courses(
    ID  varchar(8),
    Title  varchar(60),
    C-H  numeric(3,1) check (C-H>0.5 And C-H<4.0),
    Offr. Dept  varchar(10),
    Foreign key (offDept) References Dept(Deptid )
```

i+ii

ii) Note that the table **Department** has the Head of Dept field that stores only the present head of the department. It has no way to store the history of heads of each department. How can you include this feature? Explain.

iii) Students should be able to take a number of courses in each semester. First comment on the cardinality between students and courses. Finally present the DDL in this regard.

ii

1. Use a Separate History Table:

Store historical data in a separate table. For instance, create a `department_head_history` table where you log historical changes whenever a new teacher becomes the head of the department. The `teachers` table will only store the current head of the department, and historical information can be retrieved from the `department_head_history` table if needed.

2. Include a Timestamp:

Add a `timestamp` column in the `teachers` table to indicate when the current head of the department assumed the role. When assigning a new head of the department, update both the `head_of_dept` and the `timestamp` columns. This way, you can always identify the current head of the department based on the most recent timestamp.

sql — Copy code

```sql
CREATE TABLE teachers (
    teacher_id INT PRIMARY KEY,
    teacher_name VARCHAR(255),
    head_of_dept BOOLEAN, -- 1 for head of department, 0 otherwise
    timestamp TIMESTAMP, -- Timestamp indicating when the current head of th
    -- other columns
);
```

When a new head of the department is appointed:

sql — Copy code

```sql
UPDATE teachers SET head_of_dept = 0 WHERE head_of_dept = 1; -- Reset previ
INSERT INTO teachers (teacher_id, teacher_name, head_of_dept, timestamp) VA
```

3. Use Application Logic:

Implement logic in your application code to ensure that only one teacher can be marked as the head of the department at a time. When a new head is appointed, update the `head_of_dept` flag for the previous head and set it to 0. Then set the flag for the new head to 1.

iii

The relationship between students and courses in this context appears to be a many-to-many relationship. This means that each student can enroll in multiple courses, and each course can have multiple students. To model a many-to-many relationship in a relational database, you typically use an intermediary table, often called an associative or junction table. This table resolves the many-to-many relationship by storing pairs of primary keys from the related tables.

Here's how you might structure the DDL (Data Definition Language) for the `students`, `courses`, and the intermediary table, let's call it `enrollments`:

sql — Copy code

```sql
CREATE TABLE students (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(255),
    -- other student attributes
);

CREATE TABLE courses (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(255),
    -- other course attributes
);

CREATE TABLE enrollments (
    enrollment_id INT PRIMARY KEY,
    student_id INT,
    course_id INT,
    semester VARCHAR(20), -- Assuming you want to store t
    FOREIGN KEY (student_id) REFERENCES students(student_
    FOREIGN KEY (course_id) REFERENCES courses(course_id)
);
```

4]

a)

Consider the table definitions of Question No. 3 and answer the followings using standard SQL query:

i) List of the teachers name, date of birth and his/her immediate boss's name (if any) according to Salary in decreasing order.

ii) Find the students name and his/her department name whose names start with 'A' and end with 'A'.

iii) List the department name and total number of students admitted of the corresponding department.

iv) List the department name and total number of students admitted of the corresponding department for the departments whose budgets are more than 90000.

v) List the department name and total number of students admitted of the corresponding department for those departments that have at least 100 students admitted.

i]

```sql
SELECT t1.name AS teacher_name,
       t1.dob AS teacher_dob,
       t2.name AS boss_name
FROM teachers t1
LEFT JOIN teachers t2 ON t1.immediate_boss = t2.id
ORDER BY t1.salary DESC;
```

ii] select students_name, dept_name
from Students
WHERE students_name LIKE 'A%A';

iii]

```sql
SELECT d.dept_name AS department_name, COUNT(s.student_id)
FROM dept d
LEFT JOIN students s ON d.dept_id = s.dept_id
GROUP BY d.dept_name;
```

iv)

```sql
SELECT d.dept_name AS department_name, COUNT(s.student_id) as student_na
FROM dept d
LEFT JOIN students s ON d.dept_id = s.dept_id
GROUP BY d.dept_name
HAVING SUM(d.budget) > 90000;
```

v)

```
SELECT d.dept_name AS department_name, COUNT(s.student_id)
FROM dept d
LEFT JOIN students s ON d.dept_id = s.dept_id
GROUP BY d.dept_name
HAVING Count(S. student_id) >= 100;
```

b) What is a view? What is the basic difference between a table and a view? "Most SQL implementations allow updates only on simple views" Justify it using suitable example

A view in a database is a virtual table that is based on the result of a SELECT query. Unlike a physical table, a view does not store the data itself but provides a way to represent data stored in one or more tables in a specific, predefined manner. Views can simplify complex queries, provide security by restricting access to specific columns or rows, and offer a logical abstraction of the underlying data, making it easier for users to interact with the database.

The basic differences between a table and a view are as follows:

Data Storage:

Table: Tables store data physically on disk or in memory.

View: Views do not store data themselves; they are derived from the data stored in tables or other views.

Data Modification:

Table: Tables allow insertion, updating, and deletion of data using SQL DML (Data Manipulation Language) statements (INSERT, UPDATE, DELETE).

View: Views can be read-only, or they can allow limited data modification, depending on the complexity of the view. Most SQL implementations allow updates only on simple views.

Schema:

Table: Tables have a fixed schema defined by their columns and data types.

View: Views have a flexible schema defined by the SELECT statement. They can include calculations, aggregations, and joins, allowing for a more complex and customized representation of data.

Regarding the statement "Most SQL implementations allow updates only on simple views," this means that views that meet certain criteria can be updatable. These criteria often include:

The view must consist of a single table.

The view must not contain any aggregate functions (such as SUM or COUNT).

The view must not have any DISTINCT keyword.

The view must not have any GROUP BY or HAVING clauses.

The view must not have any SET operations (UNION, INTERSECT, or EXCEPT).

These restrictions ensure that the underlying data modifications through the view are unambiguous and can be directly mapped back to the original table(s). More complex views, such as those involving joins or aggregations, might not be directly updatable because it could be unclear how changes in the view should propagate back to the underlying data sources. These rules vary slightly across different database management systems, so it's essential to consult the documentation of the specific database system you are using for precise details.